What's the Problem?

What's the big deal if I want to use normal, ordinary ActiveRecord#create in my tests? I use it in my code. What could go wrong?

Since you asked …

I'll start with a simple test involving two users:

```
it "can tell which user is older" do
  eldest_user = User.create(date_of_birth: '1971-01-22')
  youngest_user = User.create(date_of_birth: '1973-08-31')
  expect(User.eldest).to eq(eldest_user)
  expect(User.youngest).to eq(youngest_user)
end
```

That test is deliberately simple so as not to distract from the data-creation issue. The only weird thing here is that I'm testing a hypothetical finder method, eldest, that goes into the database, so for the test it's necessary that the objects I create come from the database.

You make the test pass and forget about it. The test silently continues to pass every time you run your test suite.

And then …

You add authentication. And even though this test has nothing to do with authentication, it fails. Instead of returning eldest_user, the eldest call in the first assertion returns nil.

The problem is that adding authentication adds two new validations: requirements that a user must have an email address and a password. The test data no longer matches those requirements, so the objects aren't saved to the database, and therefore the finder methods can't find them—hence the nil.

With a heavy sigh, I add the required fields to the test:

```
it "can tell which user is older" do
  eldest_user = User.create!(date_of_birth: '1971-01-22',
    email: "eldest@example.com", password: "password")
  youngest_user = User.create!(date_of_birth: '1973-08-31'
    email: "youngest@example.com", password: "password")
  expect(User.eldest).to eq(eldest_user)
  expect(User.youngest).to eq(youngest_user)
end
```

Okay, that's not horrible. It's not great, either, but life marches on. I've switched to create! so now at least any further validation will fail at the point of creation, which makes diagnosing the failure much easier.